This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Improved Mean Flows: On the Challenges of Fastforward Generative Models

Zhengyang Geng[1,2,3,*] Yiyang Lu[4,2,*] Zongze Wu[3] Eli Shechtman[3] J. Zico Kolter[1] Kaiming He[2]

[1]CMU [2]MIT [3]Adobe [4]THU

Abstract

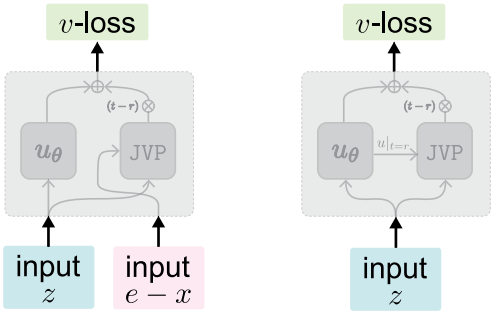
*MeanFlow (MF) has recently been established as a framework for one-step generative modeling. However, its "fastforward" nature introduces key challenges in both the training objective and the guidance mechanism. First, the original MF's training target depends not only on the underlying ground-truth fields but also on the network itself. To address this issue, we recast the objective as a loss on the instantaneous velocity v, re-parameterized by a network that predicts the average velocity u. Our reformulation yields a more standard regression problem and improves the training stability. Second, the original MF fixes the classifier-free guidance scale during training, which sacrifices flexibility. We tackle this issue by formulating guidance as explicit conditioning variables, thereby retaining flexibility at test time. The diverse conditions are processed through in-context conditioning, which reduces model size and benefits performance. Overall, our **improved MeanFlow (iMF)** method, trained entirely from scratch, achieves **1.72** FID with a single function evaluation (1-NFE) on ImageNet 256×256. iMF substantially outperforms prior methods of this kind and closes the gap with multi-step methods while using no distillation. We hope our work will further advance fastforward generative modeling as a stand-alone paradigm.*

1. Introduction

Diffusion models [39, 16, 42] and their flow-based variants [27, 28, 1] are highly effective for generative modeling. These models can be viewed as solving a differential equation (*e.g.,* an ODE) that maps a prior distribution to the data distribution. Because these equations are typically solved using multi-step numerical solvers, the generation process requires a certain number of function evaluations (NFEs).

Recently, encouraging progress [43, 40, 11, 29, 10, 57, 12] has been made toward reducing sampling steps in diffusion/flow-based models. Using the concept from physical simulation, these models can be thought of as *fastforward* approximations to the underlying differential equations. The resulting fastforward models are capable of generating in a very few or even one step. To achieve this goal, the training objectives are formulated as look-ahead mappings that operate across large time intervals, and various approximations have been proposed to address this challenging problem.

In this work, we take a deeper look at the recently proposed MeanFlow (MF) framework [12]. In MF, instead of learning the *instantaneous velocity* field (denoted by v) underlying the ODE, the model learns an *average velocity* field (denoted by u) across time steps. To avoid infeasible integration during training, MF reformulates the problem into a differential relation between the instantaneous and average velocity fields. This relation, called the "MeanFlow identity" [12], establishes a trainable objective. The underlying average velocity field serves as the ground-truth and as the optimum of this training objective.



(a) Original MeanFlow (b) Improved MeanFlow

Figure 1. **Conceptual comparison**. Original MeanFlow (MF) [12] predicts *average velocity* u by a network u_θ. As the ground-truth u is unknown, original MF substitutes u with the network's own prediction. We show that the original MF objective is equivalent to a loss on the *instantaneous velocity* v (namely, v-loss), but re-parameterized by the neural network u_θ (namely, u-pred), as shown in **(a)**. This re-parameterization, encompassed within the gray box, is determined by *the MeanFlow identity* [12]. This reformulation reveals that the input to the compound function (in the gray box) is not only the noisy data (here, z), but also the conditional velocity $(e - x)$, which is not a standard regression problem. In **(b)**, our improved objective is conceptually ***v-loss re-parameterized by u-pred***, taking only the legitimate input z.

*Equal contribution. Part of this work was done when Z. Geng was interning at Adobe and MIT, and when Y. Lu was interning at MIT.

Despite the encouraging results of the original MF [12], we identify two major issues that remain unresolved: (i) the training target in the original MF is network-dependent and therefore does not constitute a standard regression problem; (ii) MF handles the classifier-free guidance (CFG) [15] using a fixed training-time guidance scale, which sacrifices flexibility. We analyze these issues and present our solutions.

First, the original MF predicts the average velocity u, an unknown quantity that is substituted with the network's own prediction u_θ. To have a network-agnostic prediction target, we show that the original MF can be equivalently reformulated as a loss on the instantaneous velocity (namely, v-loss), which is *re-parameterized* by the network that predicts the average velocity u (namely, u-pred). See Fig. 1**(a)**. This reformulation provides a regression target v that does not depend on the network. From this perspective, we further propose to reformulate the regression input, enforcing it to depend only on the noisy sample but not on other unknown quantities (Fig. 1**(b)**). Our improved objective substantially stabilizes the training process in practice.

Second, the original MF handles CFG [15] using a *fixed* guidance scale that is determined before training. We suggest that fixing the guidance compromises the flexibility at inference time, and that the optimal value depends on the model's capability. To address this, we reformulate the guidance as a form of *conditioning*, allowing it to take varying values during both training and inference. This formulation unlocks the power and flexibility of CFG while still maintaining the 1-NFE sampling behavior. We further design an improved architecture that accommodates this and other types of conditions through in-context conditioning.

Overall, our experiments show that our **improved MeanFlow** (**iMF**) effectively addresses these issues in the original MF. In the challenging setting of 1-NFE generation on ImageNet 256×256 trained from scratch, iMF achieves an FID of **1.72**, representing a relative 50% improvement over the original MF and setting a new state-of-the-art of its kind. Our models do not use distillation or any pre-trained models for alignment. This result substantially narrows the gap with those of multi-step methods, suggesting that fastforward generative models can be a promising stand-alone framework.

2. Related Work

Diffusion and Flow-based Models. Diffusion models [39, 16, 21, 41, 42] and flow matching [27, 28, 2] lay the foundation for a series of modern generative methods. These approaches can be formulated as learning a probabilistic trajectory, *i.e.,* an ODE/SDE (ordinary/stochastic differential equation) that maps between distributions. A network is trained to model the underlying trajectory using a regression loss. Samples are generated by solving the resulting ODE or SDE, typically using a numerical solver.

Fastforward Generative Models. Standard diffusion and flow-based models were originally designed without explicitly considering the acceleration of ODE/SDE solving. An emerging category of methods, which we abstract as "fastforward generative models", explicitly incorporates ODE/SDE acceleration into their training objectives.

These models typically operate by making large jumps across time steps. Consistency Models [43, 40, 11, 29] formulate it as leaping from an intermediate time step directly to the end point of the trajectory. Consistency Trajectory Models [22] aim to learn a trajectory between any two time steps, based on explicit integration (*i.e.,* ODE/SDE solving) during training. Flow Map Matching [3] formulates the regression of the zeroth- and first-order derivatives of these flow fields. Shortcut Models [10] are built on the relationship between two time steps and their midpoint. IMM [57] leverages moment matching at different time steps. MeanFlow [12] formulates and parameterizes the average velocity across two arbitrary time steps.

Several improvements have been made to the MeanFlow formulation. AlphaFlow [55] decomposes the MeanFlow objective and adopts a schedule to interpolate from Flow Matching to MeanFlow. Decoupled MeanFlow [25] finetunes pre-trained Flow Matching models into MeanFlow by conditioning the final blocks of the networks on a second timestep. CMT [19] introduces mid-training using fixed explicit regression targets supplied by a pre-trained Flow Matching model before training the fastforward models. Our iMF is focused on the fundamental limitations of the MeanFlow objective, as well as the practical issue of CFG. These issues are orthogonal to other concurrent improvements.

3. Background

Flow Matching. Flow Matching (FM) [27, 28, 1] learns a velocity field that flows between a prior distribution and the data distribution. We consider the standard linear schedule $z_t = (1-t)\,x + t\,e$ with data $x \sim p_{\text{data}}$ and noise $e \sim p_{\text{prior}}$ (*e.g.,* Gaussian). Computing the time-derivative gives a *conditional* velocity $v_c = e - x$. Flow Matching learns a network v_θ to regress v_c by minimizing a loss function in the v-space (namely, v-loss):

$$\mathbb{E}_{t,x,e} \big\| \, v_\theta(z_t, t) - (e - x) \, \big\|^2. \tag{1}$$

As one z_t can be given by multiple pairs of (x, e), the underlying unique regression target is the *marginal velocity* [27]:

$$v(z_t, t) \triangleq \mathbb{E}[\, v_c \mid z_t \,], \tag{2}$$

which is marginalized over all pairs (x, e) that satisfy z_t at t. For brevity, we omit t and denote $v(z_t, t)$ as $v(z_t)$, and $v_\theta(z_t, t)$ as $v_\theta(z_t)$, in the remaining of this paper.

At generation time, FM samples by solving an ODE: $\frac{d}{dt} z_t = v_\theta(z_t)$. This is done by a numerical solver (*e.g.,* Euler or Heun) integrating from $t = 1$ to 0, with $z_1 \sim p_{\text{prior}}$.

MeanFlow. By viewing $v(z_t)$ as the *instantaneous* velocity field, MeanFlow (MF) [12] introduces the *average* velocity field between two time steps r and t:

$$u(z_t, r, t) \triangleq \frac{1}{t-r}\int_r^t v(z_\tau)\, d\tau. \tag{3}$$

Again, for brevity, we omit r and t and simply denote $u(z_t, r, t)$ by $u(z_t)$. Directly integrating Eq. (3) at training time is intractable. Instead, MF takes the derivative *w.r.t.* t and obtains a *MeanFlow identity* [12]:

$$u(z_t) \;=\; v(z_t) \;-\; (t-r)\,\frac{d}{dt}\,u(z_t), \tag{4}$$

which is used to establish a feasible training objective. The term $\frac{d}{dt}u$ is given by [12]:

$$\frac{d}{dt}u(z_t) = \partial_z u(z_t)\, v(z_t) + \partial_t u(z_t) \triangleq \texttt{JVP}(u; v). \tag{5}$$

This can be computed by Jacobian-vector product (JVP), between the Jacobian $[\partial_z u, \partial_r u, \partial_t u]$ and a tangent vector $[v, 0, 1]$. Here, for brevity, we introduce the notation $\texttt{JVP}(u; v)$ for this JVP computed at $u(z_t)$ and $v(z_t)$.

MeanFlow *parameterizes* average velocity by a network $u_\theta(z_t)$ (conditioned on r and t, omitted in notation for brevity). This network is optimized to approximate the MeanFlow identity (4). In the formulation of the original MF, Eq. (4) is implemented as:

$$u_{\text{tgt}} = (e - x) - (t - r)\,\texttt{JVP}(u_\theta; e - x), \tag{6}$$

Here, two approximations are made [12]: (i) the marginal $v(z_t)$ is replaced with the conditional $v_c = e - x$, same as Flow Matching; (ii) the true u in JVP is replaced by its network prediction u_θ. With this target u_{tgt}, MF optimizes:

$$\mathbb{E}_{t,r,x,e}\, \|u_\theta - \text{sg}(u_{\text{tgt}})\|^2, \tag{7}$$

where "sg" denotes stop-gradient, which helps create an *apparent* target for training. Once trained, MF directly performs one-step sampling via $z_0 = z_1 - u_\theta(z_1)$ given $(r, t) = (0, 1)$, with $z_1 \sim p_{\text{prior}}$.

4. Improved Mean Flows

We identify and address two challenges of the original MF model. **(i)** The apparent target in Eq. (6) depends on the network. We aim for a more standard regression formulation (Sec. 4.1). **(ii)** To extend the MeanFlow identity for supporting CFG, original MF fixes the guidance scale before training. We relax this constraint and allow for flexible CFG (Sec. 4.2). We then introduce an improved architecture for handling many types of conditions by in-context conditioning (Sec. 4.3).

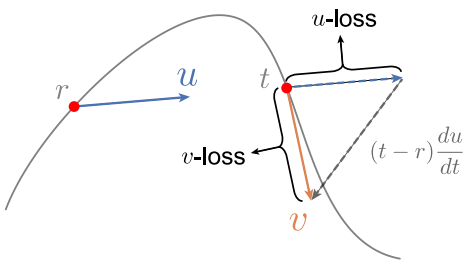


Figure 2. **MeanFlow as v-loss.** Original MeanFlow (MF) [12] models the average velocity u and train the network u_θ via a **u-loss** parameterized by u_θ itself. We show that MF can be reformulated as a **v-loss** re-parameterized by u_θ, driven by the MeanFlow identity in Eq. (8).

4.1. MeanFlow as v-loss

Eq. (7) suggests that original MF [12] is a **u-loss** parameterized by **u-pred**. In this subsection, we first show that the original MF can be reformulated as a **v-loss** (*i.e.,* instantaneous velocity) *re-parameterized* by **u-pred**. This gives us a network-independent target. See Fig. 2.

This reformulation reveals a hidden issue: the prediction function for v requires access to *unknown* quantities, not just z_t. We provide a solution to remedy this issue. With our reformulation, we arrive at a more standard regression problem.

Reformulating MeanFlow as v-loss. While MF aims to compute u-loss, the true target u is not accessible. As a result, the target u_{tgt} has a term approximated by $\texttt{JVP}(u_\theta; e - x)$ in Eq. (6), which is not a standard regression target. We notice that the *instantaneous* velocity v can serve as a more feasible target. We can rewrite the MeanFlow identity (4) as (see also Fig. 2):

$$v(z_t) \;=\; u(z_t) \;+\; (t-r)\,\frac{d}{dt}\,u(z_t). \tag{8}$$

Here, v on the left-hand side can serve as a target, as in standard Flow Matching; the *compound function* on the right-hand side can be parameterized by u_θ. We denote the (re-)parameterized compound function as V_θ:

$$V_\theta \triangleq u_\theta\,(z_t) + (t - r)\,\texttt{JVP}_{\text{sg}}(u_\theta; e - x), \tag{9}$$

where $\texttt{JVP}_{\text{sg}}$ denotes stop-gradient on the $\frac{d}{dt}u_\theta$ outcome (we will discuss stop-gradient later). Then we obtain a Flow Matching-like objective function, similar to Eq. (1):

$$\mathbb{E}_{t,r,x,e}\|V_\theta - (e - x)\|^2. \tag{10}$$

It is easy to show that the reformulation in (9)(10) is fully *equivalent* to the original MF objective in (6)(7). This suggests that *MeanFlow can be viewed as v-loss re-parameterized by u_θ*. Such re-parameterization in Eq. (9) is driven by the MeanFlow identity in Eq. (8).

Algorithm 1 improved MeanFlow: training.

Note: in PyTorch and JAX, `jvp` returns the function output and JVP.

```
# fn(z, r, t): function to predict u
# x: training batch

t, r = sample_t_r()
e = randn_like(x)

z = (1 - t) * x + t * e

# instantaneous velocity v at time t
v = fn(z, t, t)

# predict u and dudt
u, dudt = jvp(fn, (z, r, t), (v, 0, 1))

# compound function V
V = u + (t - r) * stopgrad(dudt)
error = V - (e - x)

loss = metric(error)
```

This reformulation reveals a new issue: V_θ in Eq. (9) does not only take z_t as input, but more importantly, also takes $e - x$ as another input. Formally, our parameterized compound function V_θ is:

$$V_\theta(z_t, e - x). \tag{11}$$

This is illustrated in Fig. 1**(a)**. From the perspective of a standard regression formulation (*e.g.,* Eq. (1)), this is not a fully legitimate prediction function. We will show the negative effect of this extra input in Fig. 3.

Improved MeanFlow Parameterization. The reason for V_θ's dependence on $e - x$ is on JVP, which can be traced back to the approximation in Eq. (6): the *marginal* velocity v in Eq. (5) is replaced by the *conditional* velocity $v_c = e - x$. Rather than doing this replacement, we can parameterize the marginal v instead. Formally, we re-define the compound function V_θ as:

$$V_\theta\left(z_t\right) \triangleq u_\theta\left(z_t\right) + (t - r)\,\mathrm{JVP}_{\mathrm{sg}}\left(u_\theta; v_\theta\right). \tag{12}$$

Here, inside the function of JVP, both u_θ and v_θ are network predictions: both take z_t as the sole input. As such, our V_θ takes only z_t as the input, which is thus a legitimate prediction function. See Fig. 1**(b)**.

To realize v_θ with minimal overhead, we can reuse all or most of the network u_θ. We propose two solutions:

- **Boundary condition** of u_θ. By definition, we have the relation: $v(z_t, t) \equiv u(z_t, t, t)$, that is, v equals u at $r \to t$. As such, we can simply represent $v_\theta(z_t, t)$ by $u_\theta(z_t, t, t)$. This solution introduces no extra parameters. We empirically show that this is sufficient for addressing the issue we consider here.

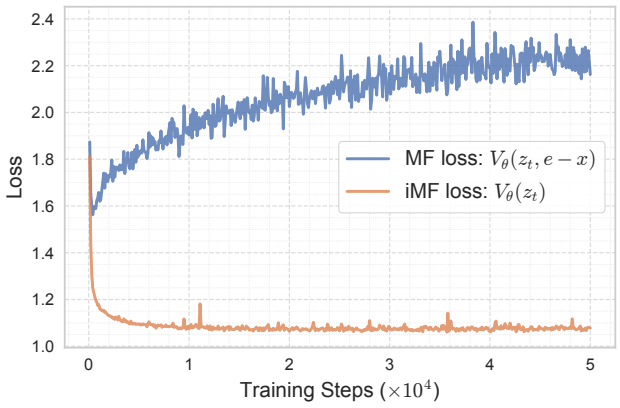


Figure 3. **Training losses.** We examine the loss of samples only with $t \neq r$, since a batch also contains samples of $t = r$, for which the JVP term becomes zero due to its coefficient $(t - r)$. Both MF and iMF can be viewed as v-loss, using different forms of compound V_θ. Original MF's loss is non-decreasing and has high variance. (Settings: MeanFlow-B/2, trained with basic ℓ_2 loss with no adaptive weighting, and with no CFG.)

- **Auxiliary v-head.** Beyond directly reusing $u_\theta(z_t, t, t)$, we can add an auxiliary head in the network of $u_\theta(z_t, r, t)$ that serves as a subnetwork v_θ. This introduces extra capacity for modeling v. It is at the cost of extra *training-time* parameters, which, however, are not used at inference-time (as only u_θ is needed). More details are in appendix. Using this head improves the results further.

The pseudo-code of our iMF formulation is in Alg. 1, where the simpler form of $v_\theta(z_t, t) \equiv u_\theta(z_t, t, t)$ is shown.

Comparison and Analysis. In Fig. 3, we compare the training loss between the original MF and the iMF objectives (*without* auxiliary v-head). We examine only the samples with $t \neq r$, as the JVP term becomes zero when $t = r$ and thus is not our focus.

Although the two formulations only differ in $V_\theta(z_t, e-x)$ and $V_\theta(z_t)$, this distinction results in strikingly different behavior. The original MF's loss has a much *higher variance* and is non-decreasing[1], even though its objective can still successfully enable one-step generation.

This comparison may look counterintuitive, because the form of $V_\theta(z_t, e - x)$ seems to "leak" the regression target. However, we note that the true, unique regression target for v-loss is not the conditional velocity $e - x$, but the marginal $v(z_t) = \mathbb{E}[\, v_c \mid z_t \,]$ in Eq. (2), and therefore the leaking does not directly disclose the true $v(z_t)$. On the other hand, according to Eq. (5), the input to JVP should not be the conditional $v_c = e - x$, but should be the marginal $v(z_t)$. As this is the input tangent vector to JVP, the variance of the conditional velocity can be significantly magnified by

[1] If we also include the samples of $t = r$, the original MF's overall loss can still decrease, depending on the portion of such samples.

JVP (*i.e.,* the Jacobian-*vector* product). Our tangent vector is predicted by $v_\theta(z_t)$, which should have lower variance than $e - x$. Fig. 3 suggests that the large variance dominates the resulting loss.

About Stop-gradient. Our formulation does not remove the stop-gradient operation, as indicated by the notation $\texttt{JVP}_{\text{sg}}$. Unlike original MF, in our case the stop-gradient is part of the prediction function V_θ, not the regression target. As such, in principle, this stop-gradient is not strictly needed for the formulation itself. However, in practice, we observe that using the stop-gradient inside V_θ is still beneficial, as removing it introduces high-order gradients *w.r.t.* θ and makes optimization more difficult.

4.2. Flexible Guidance

Thus far, we have not discussed the formulation of classifier-free guidance (CFG) [15]. The original MF [12] proposed a formulation to support 1-NFE CFG, provided that a guidance scale is *fixed* at training-time. However, a fixed guidance scale sacrifices the flexibility of adjusting this core hyperparameter at inference time. More importantly, the optimal CFG scales *shift* under different settings (Fig. 4), and in general, a *strong* model (*e.g.,* larger size, longer training, and/or more NFEs) favors a *smaller* CFG scale. It is suboptimal to freeze the scale *a priori*.

To address this issue, we reformulate the CFG scale as a form of *conditioning*, analogous to how a model is conditioned on time steps (*e.g.,* t and r). This enables the scale to vary at training and inference time.

Original MeanFlow with fixed guidance. The original MF [12] considers a *fixed* guidance field v_{cfg}:

$$v_{\text{cfg}}(z_t \mid \mathbf{c}) = \omega\, v(z_t \mid \mathbf{c}) + (1-\omega)\, v(z_t), \tag{13}$$

where $\mathbf{c}$ is the class-condition, and ω is a fixed guidance scale. Combining this definition and the MeanFlow identity, the original MF learns a *class-conditional* average velocity, namely, $u_\theta(z_t \mid \mathbf{c})$. We omit the derivations here and refer readers to [12]; but analogous to our reformulation in Sec. 4.1, conceptually, we can re-parameterize $u_\theta(z_t \mid \mathbf{c})$ into a compound function:

$$V_\theta(\cdot \mid \mathbf{c}) \triangleq u_\theta(z_t \mid \mathbf{c}) + (t-r)\, \texttt{JVP}_{\text{sg}}. \tag{14}$$

Here, for brevity, we omit the input to V_θ (and to JVP), which is not our focus in this subsection: our discussion here can support both objectives in Eq. (9) (original MF) and Eq. (12) (iMF). This V_θ is trained to fit a target determined by a fixed ω in the original MF [12].

Improved MeanFlow with flexible guidance. If the underlying guidance field v_{cfg} (13) is given by different ω values, we can still let our neural network fit each of them. To do so, we only need to allow the network to *condition* on the CFG guidance scale ω. Similar strategies have been studied in multi-step methods [31, 6, 46], which we extend to our one-step method here.

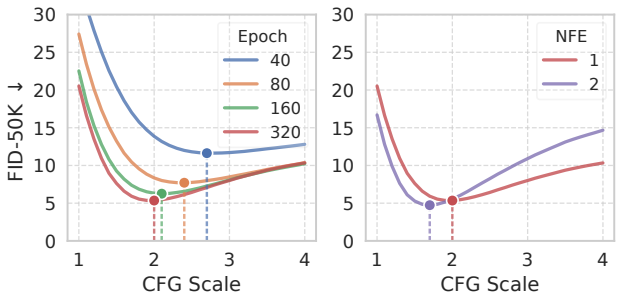


Figure 4. **Optimal CFG scales shift under different settings**. In general, a *stronger* setting has a *smaller* optimal CFG scale, as reflected by increased training epochs (left) and inference steps (right). This investigation is enabled by our flexible CFG-conditioning, where a single model can support varying CFG scales even in the single/few-NFE case. (Settings: iMF-B/2 on ImageNet 256×256.)

Formally, we extend Eq. (14) to:

$$V_\theta(\cdot \mid \mathbf{c}, \omega) \triangleq u_\theta(z_t \mid \mathbf{c}, \omega) + (t-r)\, \texttt{JVP}_{\text{sg}}, \tag{15}$$

which indicates that our compound function V_θ can be conditioned on ω, and this conditioning is handled by the network u_θ. This is analogous to standard time-conditioning (*e.g.,* t and r), which turns a continuous value into a learnable embedding. At training time, the value of ω is randomly sampled from a given distribution. The implementation details of training with CFG conditioning is in appendix.

Fig. 4 shows the effect of our flexible CFG in iMF. Under different training and inference settings, the optimal guidance scale varies. Even for the *same* model, training longer or using more inference steps can favor a different guidance scale, and therefore it is impossible to find the optimal scale beforehand. Our design unlocks the full potential of CFG for 1-NFE models.

Additional guidance conditioning. Our formulation not only enables conditioning on a single variable ω, but also allows for other guidance-related factors. We can handle CFG interval [23] under the same paradigm.

CFG interval [23] is an effective technique for improving sample diversity. In its original definition, it applies CFG only to a time interval $[t_{\text{min}}, t_{\text{max}}]$ at *inference-time*. To support this behavior at training time in our one-step model, we can also view the two values t_{min}, t_{max} as a form of conditioning. At training time, when t is outside of this interval, CFG is disabled (by setting $\omega = 1$).

We use the notation $\Omega = \{\omega, t_{\text{min}}, t_{\text{max}}\}$ to denote all conditions related to CFG. Each item in Ω has its own embedding. All conditions can be handled by standard adaLN-zero, or in-context conditioning, discussed next.

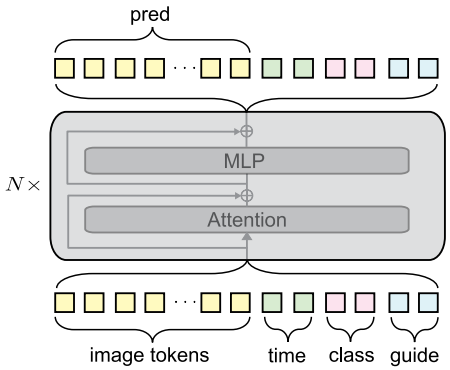


Figure 5. **Improved in-context conditioning.** Each type of conditions is turned into *multiple* tokens, which are concatenated with the image latent tokens along the sequence axis. It accommodates the conditions of time steps (r, t), class $\mathbf{c}$, and guidance-related factors Ω (CFG scale ω and CFG intervals). Importantly, ***we do not use adaLN-zero for conditioning***, which significantly reduces the model size (number of parameters) while maintaining performance.

4.3. Improved In-context Conditioning

Our model has *a diverse set* of conditions, including two time steps r and t, a class label $\mathbf{c}$, and the guidance-related conditions Ω. In its complete notation, the network u_θ is:

$$u_\theta = u_\theta\left(z_t \mid r,\, t,\, \mathbf{c},\, \Omega\right). \tag{16}$$

Typically, the conditioning is handled by adaLN-zero [32], which *sums* all condition embeddings. When many heterogeneous conditions are present, summing their embeddings and processing by adaLN-zero may become less effective, as this single operation can be overburdened.

Improved MeanFlow conditioning. To handle these many conditions, we resort to the *in-context* conditioning strategy. In-context conditioning was explored in DiT [32] but was found inferior to adaLN-zero in their setting. We find that this gap can be closed if *multiple* tokens are used for each condition. In our implementation, we use 8 tokens for class, and 4 tokens for each other conditions (see appendix). All these learnable tokens are concatenated along the sequence axis, jointly with the tokens from images (in the latent space, same as DiT [32]). The sequence is processed by Transformer blocks (Fig. 5). This architecture enables us to accommodate different types of conditions flexibly.

As an important by-product, our in-context conditioning enables us to completely *remove* adaLN-zero, which is parameter-heavy. This yields a **1/3 reduction** in model size (*e.g.,* from 133M to 89M for our iMF-Base model) when depth and width are unchanged. This also allows us to design the larger models more flexibly.

5. Experiments

Our experiment settings follow those of the original MeanFlow [12], using the same public code. The experiments are

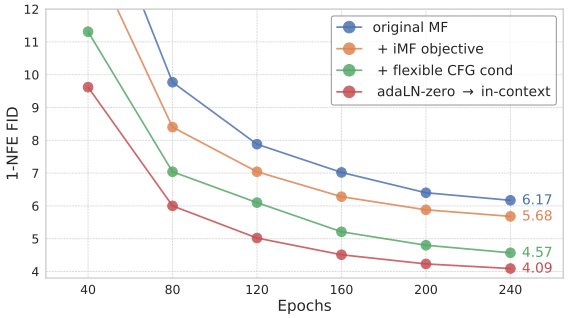


Figure 6. **FID curves during training.** The original MeanFlow-B/2 baseline has a 1-NFE FID of 6.17. Using the improved training objective (Sec. 4.1), FID improves to 5.68. Incorporating flexible CFG conditioning (Sec. 4.2) reduces FID to 4.57. Replacing adaLN-zero with in-context conditioning (Sec. 4.3) further improves FID to 4.09. See also Tab. 1.

on ImageNet [8] class-conditional generation at 256×256 resolution. Following [10, 57, 12], the model operates on the latent space of a pretrained VAE tokenizer [35], which produces 32×32×4 latents from 256×256×3 images.

We evaluate the challenging protocol of **1-NFE** generation, where all our models are trained ***from scratch***. We report Fréchet Inception Distance (FID) [14] on 50K generated images (with additional metrics in Tab. 2). Detailed configurations are in appendix.

Baseline. In our ablations, we use the MeanFlow-B/2 model [12]. The ablation models are trained for 240 epochs. Our setting is exactly the same as that of MeanFlow-B/2 in [12], which has a 1-NFE FID of 6.17 (with CFG). This model is our starting point.

5.1. Ablation Study

In Tab. 1**(a)** and **(b)**, we ablate the iMF designs discussed in Sec. 4.1 and Sec. 4.2. The architectural improvements are in Tab. 1(c). FID curves during training are in Fig. 6.

MeanFlow as v-loss. In Tab. 1**(a)**, we compare the original MF training formulation (7) with our iMF training formulation (12). We do not use CFG-conditioning here. We compare two variants of computing v_θ for the `JVP` usage in (12): the boundary condition or auxiliary head.

When using the variant of boundary condition ($v_\theta = u_\theta(z_t, t, t)$), our formulation improves the case w/o CFG from an FID of 32.69 to 29.42, representing a solid gain of 3.27. This variant adds no extra parameters at training or inference time. This result demonstrates the impact of the legitimate regression formulation.

When using the auxiliary head variant, our formulation also substantially improves over the original MF, from 32.69 to 30.76 w/o CFG, with a gain of 1.93. While the gain is smaller than that of using the boundary condition, it becomes relatively more significant in the case of "w/ CFG", suggesting a more capable model is desired to handle the more challenging scenario.

FID, 1-NFE	w/o CFG	w/ CFG
original MF [12]	32.69	6.17
our V_θ, with $v_\theta = u_\theta(z_t, t, t)$	**29.42**	5.97
our V_θ, with v_θ from aux. head	30.76	**5.68**

(a) **MeanFlow as v-loss**. We compare with original MF with our iMF objective in Eq. (12) in Sec. 4.1. We compare two variants of computing v_θ for Eq. (12), namely, using u_θ's boundary condition or an auxiliary head. In each row, "w/o CFG" and "w/ CFG" are two models trained separately, as is in original MF (which does not support flexible inference-time CFG).

FID, 1-NFE	w/o CFG	w/ CFG
best in Tab. 1**(a)**	30.76	5.68
CFG-condition: ω-condition	25.15	5.52
CFG-condition: Ω-condition	**20.95**	**4.57**

(b) **Flexible guidance**. Adding guidance as conditioning enables flexible guidance at inference (Sec. 4.2). ω-condition is the basic conditioning on the guidance scale ω. Ω-condition allows to further condition on CFG interval's start and end points. Here, only in the first row, "w/o CFG" and "w/ CFG" are two models trained separately, same as Tab. 1**(a)**; when using our flexible CFG (last two rows), the "w/o CFG" cases are simply $\omega = 1$ at inference-time, using a single trained model.

FID, 1-NFE	# params	w/ CFG
best in Tab. 1**(b)**	133M	4.57
adaLN-zero $\rightarrow$ in-context cond.	**89M**	**4.09**
+ advanced Transformer blocks	89M	3.82
+ longer training (640ep)	89M	3.39

(c) **In-context conditioning** and other improvements. Replacing adaLN-zero [32] with our multi-token in-context conditioning (Sec. 4.3) improves the results and substantially reduces the model size. Advanced Transformer blocks and longer training yield improvements as expected.

Table 1. **Ablation study on 1-NFE generation.** FID-50K is evaluated on ImageNet 256×256. All are with the MF-B/2 backbone, trained for 240 epochs from scratch by default.

In the case of "w/ CFG" (here, trained with a fixed ω), using the boundary condition improves FID from 6.17 to 5.97 (Tab. 1**(a)**). While this relative gain is smaller, we observe that the same setting has a more pronounced impact on the same MF-XL model:

FID, 1-NFE	**MF-XL/2 model**, w/ CFG
original MF [12]	3.43
our V_θ, with $v_\theta = u_\theta(z_t, t, t)$	**2.99**

We hypothesize that when the model has more capacity, it can better leverage the capacity to learn v_θ by $u_\theta(z_t, t, t)$, and therefore benefits more from this formulation.

Further, Tab. 1**(a)** shows that the auxiliary head achieves an FID of 5.68 w/ CFG, which is about 10% relative improvement over the original MF. This auxiliary head introduces no extra parameters or compute at inference time. All these comparisons demonstrate that a reliable v estimation *as* `JVP`*'s input* is critical for MeanFlow methods.

Flexible guidance. In Tab. 1**(b)**, we examine the CFG conditioning proposed in Sec. 4.2. This ablation is best examined together with Fig. 4: the major advantage of CFG conditioning is on the inference-time flexibility, which may not be simply reflected by a single FID number.

config	depth	width	# params	Gflops	FID↓	IS↑
MF-B/2	12	768	131M	23.1	6.17	208.0
MF-M/2	16	1024	308M	54.0	5.01	252.0
MF-L/2	24	1024	459M	80.9	3.84	250.9
MF-XL/2	28	1152	676M	119.0	3.43	247.5
iMF-B/2	12	768	89M	24.9	3.39	255.3
iMF-M/2	24	768	174M	49.9	2.27	257.7
iMF-L/2	32	1024	409M	116.4	1.86	276.6
iMF-XL/2	48	1024	610M	174.6	**1.72**	**282.0**

Table 2. **System-level comparison with original MeanFlow**, evaluated by FID and IS [36] on ImageNet 256×256 with **1-NFE** generation. The notations of B/M/L/XL are mainly for reference, as it is impossible to calibrate both model size (# params) and compute (Gflops) due to the removal of adaLN-zero. The compute is for 1-NFE of the generator, excluding the tokenizer decoder.

In Tab. 1**(b)**, when using the simpler ω-conditioning (*i.e.,* only on the CFG scale ω), the FID w/ CFG improves slightly from 5.68 to 5.52. This marginal gain is unsurprising, because the original MF [12] already had a near-optimal but fixed training-time ω, for this small model. This gain is more substantial for larger models, for which searching for a fixed training-time ω becomes impractical.

Tab. 1**(b)** further shows that richer CFG-conditioning (*i.e.,* on Ω) substantially improves the FID, by 1.11 to 4.57. This gain is because Ω-conditioning enables CFG interval [23] *at inference time*, and CFG interval is highly effective even for multi-step methods. Our conditioning strategy does not affect the 1-NFE sampling behavior: $(t_{\text{min}}, t_{\text{max}})$ are turned into embeddings for 1-NFE generation.

Interestingly, our CFG conditioning also enables us to mimic the "w/o CFG" behavior at inference time. We achieve this by setting $\omega = 1.0$ at inference, which represents the "no CFG" case (see Eq. (13)). Tab. 1**(b)** shows that the FID at $\omega = 1.0$ ("w/o CFG") is significantly improved by 10 points, from 30.76 to 20.95. This suggests that training our models across a range of CFG scales can improve their *generalization* performance, substantially improving their results even at a suboptimal ω value.

In-context conditioning. Thus far, our ablations have been using adaLN-zero [32] for conditioning. In Tab. 1**(c)**, we replace it with our multi-token in-context conditioning (Sec. 4.3). As adaLN-zero is parameter-heavy, removing it yields a substantial 1/3 reduction in model size, from 133M to 89M. Such a reduction is highly attractive for larger models. The FID is improved from 4.57 to 4.09.

Finally, following [51], we incorporate general-purpose Transformer improvements: SwiGLU [38], RMSnorm [54], and RoPE [44]. These components put together improve FID from 4.09 to 3.82. Training longer yields an extra gain, achieving 3.39 FID with this B-size model.

5.2. Comparisons with Original MeanFlow

In Tab. 2, we provide a system-level comparison with the original MF [12]. We note that removing adaLN-zero makes

Method	# Params	NFE	FID
1-NFE diffusion/flow from scratch			
iCT-XL/2 [40]	675M	1	34.24
Shortcut-XL/2 [10]	675M	1	10.60
MeanFlow-XL/2 [12]	676M	1	3.43
TiM-XL/2 [50]	664M	1	3.26
α-Flow-XL/2+ [55]	676M	1	2.58
iMF-B/2 (ours)	89M	1	3.39
iMF-M/2 (ours)	174M	1	2.27
iMF-L/2 (ours)	409M	1	1.86
iMF-XL/2 (ours)	610M	1	**1.72**
2-NFE diffusion/flow from scratch			
iCT-XL/2 [40]	675M	2	20.30
IMM-XL/2 [57]	675M	1×2	7.77
MeanFlow-XL/2+ [12]	676M	2	2.20
α-Flow-XL/2+ [55]	676M	2	1.95
iMF-XL/2 (ours)	610M	2	**1.54**

Method	# Params	NFE	FID
1-NFE diffusion/flow (distillation)			
π-Flow-XL/2 [7]	675M	1	2.85
DMF-XL/2+ [25]	675M	1	2.16
FACM-XL/2 [33]	675M	1	1.76
Multi-NFE diffusion/flow			
ADM-G [9]	554M	250×2	4.59
LDM-4-G [35]	400M	250×2	3.60
SimDiff [17]	2B	1000×2	2.77
DiT-XL/2 [32]	675M	250×2	2.27
SiT-XL/2 [30]	675M	250×2	2.06
SiT-XL/2 + REPA [53]	675M	250×2	1.42
SiD2 [18]	–	512×2	1.38
LightningDiT-XL/2 [51]	675M	250×2	1.35
DDT-XL/2 [49]	675M	250×2	1.26
RAE [56] + DiT^{DH}-XL	839M	50×2	1.13

Method	# Params	NFE	FID
GANs			
BigGAN [4]	112M	1	6.95
GigaGAN [20]	569M	1	3.45
StyleGAN-XL [37]	166M	1	2.30
autoregressive/masking			
JetFormer-L [48]	2.75B	256×2	6.64
MaskGIT [5]	227M	8	6.18
RQ-Transformer [24]	3.8B	256×2	3.80
STARFlow [13]	1.4B	1024×2	2.40
LLamaGen-3B [45]	3.1B	256×2	2.18
VAR-$d30$ [47]	2B	10×2	1.92
MAR-H [26]	943M	256×2	1.55
RAR-XXL [52]	1.5B	256×2	1.48
xAR-H [34]	1.1B	50×2	1.24

Table 3. **System-level comparison on class-conditional ImageNet 256×256**. **Left**: 1-NFE and 2-NFE diffusion/flow models trained *from scratch*. **Middle**: Diffusion/flow models, including distillation-based 1-NFE methods and multi-NFE methods. **Right**: Reference methods from other generative modeling families, including GANs and autoregressive/masking models. All numbers are with CFG when applicable, and ×2 in NFE indicates that the CFG computation doubles NFEs at inference time.

it impossible to fully calibrate the model sizes, and as such, the B/M/L/XL notations are mainly for the ease of referring. In our designs: **(i)** the B-size model has the same depth and width in both MF and iMF; **(ii)** the M-size model is designed to have *smaller* size and *less* compute; and **(iii)** the L/XL-size models are designed to roughly match the model size of the MF counterparts (yet are still ∼10% smaller).

Overall, Tab. 2 shows that our iMF models have substantially better FID and IS results. Our iMF-XL/2 model achieves a 1-NFE FID of **1.72**, representing a **50%** relative reduction compared to MF-XL/2's 3.43. Qualitative examples are in Fig. 7 and appendix.

5.3. Comparisons with Previous Methods

In Tab. 3, we provide system-level comparisons with previous methods. We categorize the methods into: (i) fastforward generative models trained *from scratch* (Tab. 3, left); (ii) fastforward generative models, *distilled* from pre-trained multi-step models (Tab. 3, mid-top); (iii) multi-step diffusion/flow models (Tab. 3, mid-bottom); (iv) GAN and autoregressive models (Tab. 3, right).

Fastforward models from scratch. Tab. 3 (left) shows that our iMF substantially outperforms other fastforward models that are also trained from scratch. In addition, our 1-NFE FID of 1.72 also outperforms those *distilled* from pre-trained models (Tab. 3, mid-top), suggesting that training from scratch can produce highly competitive fastforward models. When relaxing NFE to 2, iMF achieves an FID of **1.54**. This further closes the gap with the *many-step* diffusion/flow models (Tab. 3, mid-bottom).

6. Conclusion

We have demonstrated that fastforward generative models, without pretraining, can achieve highly competitive performance. We hope this encouraging result represents a solid step towards stand-alone fastforward generation.



class 12: house finch, linnet, Carpodacus mexicanus



class 738: pot, flowerpot



class 975: lakeside, lakeshore

Figure 7. **Qualitative results of 1-NFE generation on ImageNet 256×256.** We show *uncurated* results on the three classes listed here; more are in appendix. The model is iMF-XL/2.

With the remarkable progress of 1-NFE generation, the use of a tokenizer begins to incur a non-negligible cost at inference time. While our work focuses on advancing fastforward models and is orthogonal to tokenizer design, from a practical standpoint, reducing or removing the tokenizer is becoming increasingly valuable. We expect future research to explore efficient tokenizers or pixel-space generation.

References

[1] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *ICLR*, 2023.

[2] Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. In *ICLR*, 2023.

[3] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. Flow map matching. *TMLR*, 2025.

[4] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *ICLR*, 2019.

[5] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. Maskgit: Masked generative image transformer. In *CVPR*, 2022.

[6] Huayu Chen, Kai Jiang, Kaiwen Zheng, Jianfei Chen, Hang Su, and Jun Zhu. Visual generation without guidance. In *ICML*, 2025.

[7] Hansheng Chen, Kai Zhang, Hao Tan, Leonidas Guibas, Gordon Wetzstein, and Sai Bi. pi-flow: Policy-based few-step generation via imitation distillation. *arXiv preprint arXiv:2510.14974*, 2025.

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009.

[9] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In *NeurIPS*, 2021.

[10] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *ICLR*, 2025.

[11] Zhengyang Geng, Ashwini Pokle, William Luo, Justin Lin, and J Zico Kolter. Consistency models made easy. In *ICLR*, 2024.

[12] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. In *NeurIPS*, 2025.

[13] Jiatao Gu, Tianrong Chen, David Berthelot, Huangjie Zheng, Yuyang Wang, Ruixiang Zhang, Laurent Dinh, Miguel Angel Bautista, Josh Susskind, and Shuangfei Zhai. Starflow: Scaling latent normalizing flows for high-resolution image synthesis. *NeurIPS*, 2025.

[14] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *NeurIPS*, 2017.

[15] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop*, 2021.

[16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[17] Emiel Hoogeboom, Jonathan Heek, and Tim Salimans. simple diffusion: End-to-end diffusion for high resolution images. In *ICML*, 2023.

[18] Emiel Hoogeboom, Thomas Mensink, Jonathan Heek, Kay Lamerigts, Ruiqi Gao, and Tim Salimans. Simpler diffusion (sid2): 1.5 fid on imagenet512 with pixel-space diffusion. In *CVPR*, 2025.

[19] Zheyuan Hu, Chieh-Hsin Lai, Yuki Mitsufuji, and Stefano Ermon. Cmt: Mid-training for efficient learning of consistency, mean flow, and flow map models. *arXiv preprint arXiv:2509.24526*, 2025.

[20] Minguk Kang, Jun-Yan Zhu, Richard Zhang, Jaesik Park, Eli Shechtman, Sylvain Paris, and Taesung Park. Scaling up gans for text-to-image synthesis. In *CVPR*, 2023.

[21] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *NeurIPS*, 2022.

[22] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ODE trajectory of diffusion. In *ICLR*, 2024.

[23] Tuomas Kynkäänniemi, Miika Aittala, Tero Karras, Samuli Laine, Timo Aila, and Jaakko Lehtinen. Applying guidance in a limited interval improves sample and distribution quality in diffusion models. In *NeurIPS*, 2024.

[24] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *CVPR*, 2022.

[25] Kyungmin Lee, Sihyun Yu, and Jinwoo Shin. Decoupled meanflow: Turning flow models into flow maps for accelerated sampling. *arXiv preprint arXiv:2510.24474*, 2025.

[26] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive image generation without vector quantization. In *NeurIPS*, 2024.

[27] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *ICLR*, 2023.

[28] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *ICLR*, 2023.

[29] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. In *ICLR*, 2025.

[30] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *ECCV*, 2024.

[31] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik P Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *CVPR*, 2023.

[32] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *CVPR*, 2023.

[33] Yansong Peng, Kai Zhu, Yu Liu, Pingyu Wu, Hebei Li, Xiaoyan Sun, and Feng Wu. Flow-anchored consistency models. *arXiv preprint arXiv:2507.03738*, 2025.

[34] Sucheng Ren, Qihang Yu, Ju He, Xiaohui Shen, Alan Yuille, and Liang-Chieh Chen. Beyond next-token: Next-x prediction for autoregressive visual generation. In *ICCV*, 2025.

[35] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2021.

[36] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. In *NeurIPS*, 2016.

[37] Axel Sauer, Katja Schwarz, and Andreas Geiger. Stylegan-xl: Scaling stylegan to large diverse datasets. In *SIGGRAPH*, 2022.

[38] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[39] Jascha Sohl-Dickstein, Eric A Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.

[40] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. In *ICLR*, 2024.

[41] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019.

[42] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.

[43] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *ICML*, 2023.

[44] Jianlin Su, Yu Lu, Shengfeng Pan, Murtadha Ahmed, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[45] Peize Sun, Yi Jiang, Shoufa Chen, Shilong Zhang, Bingyue Peng, Ping Luo, and Zehuan Yuan. Autoregressive model beats diffusion: Llama for scalable image generation. *arXiv preprint arXiv:2406.06525*, 2024.

[46] Zhicong Tang, Jianmin Bao, Dong Chen, and Baining Guo. Diffusion models without classifier-free guidance. In *ICML*, 2025.

[47] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. In *NeurIPS*, 2024.

[48] Michael Tschannen, André Susano Pinto, and Alexander Kolesnikov. Jetformer: An autoregressive generative model of raw images and text. In *ICLR*, 2025.

[49] Shuai Wang, Zhi Tian, Weilin Huang, and Limin Wang. Ddt: Decoupled diffusion transformer. *arXiv preprint arXiv:2504.05741*, 2025.

[50] Zidong Wang, Yiyuan Zhang, Xiaoyu Yue, Xiangyu Yue, Yangguang Li, Wanli Ouyang, and Lei Bai. Transition models: Rethinking the generative learning objective. *arXiv preprint arXiv:2509.04394*, 2025.

[51] Jingfeng Yao, Bin Yang, and Xinggang Wang. Reconstruction vs. generation: Taming optimization dilemma in latent diffusion models. In *CVPR*, 2025.

[52] Qihang Yu, Ju He, Xueqing Deng, Xiaohui Shen, and Liang-Chieh Chen. Randomized autoregressive visual generation. In *ICCV*, 2025.

[53] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. In *ICLR*, 2025.

[54] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.

[55] Huijie Zhang, Aliaksandr Siarohin, Willi Menapace, Michael Vasilkovsky, Sergey Tulyakov, Qing Qu, and Ivan Skorokhodov. Alphaflow: Understanding and improving meanflow models. *arXiv preprint arXiv:2510.20771*, 2025.

[56] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders. *arXiv preprint arXiv:2510.11690*, 2025.

[57] Linqi Zhou, Stefano Ermon, and Jiaming Song. Inductive moment matching. In *ICML*, 2025.